

PROGRAMMAZIONE PROCEDURALE

A.A. 2025/2026

DEFINITION AND DECLARATIONS

DEFINITION AND DECLARATION

- ⌚ Defining something means providing all of the necessary information to create that thing in its entirety.
 - ✓ **Defining a function means providing a function body;**
 - ✓ **Defining a variable means allocating memory for it.**
- ⌚ Definitions are also declarations
 - ✓ Not only for functions but also for variables
- ⌚ Most of the time, when you declare a variable, you are also providing the definition. What does it mean to define a variable, exactly? It means you are telling the compiler where to create the storage for that variable.

IN SUMMARY

- ⌚ A declaration provides basic attributes of a symbol: its type and its name.
- ⌚ A definition provides all of the details of that symbol--if it's a function, what it does; if it's a variable, where that variable is stored.
- ⌚ Often, the compiler only needs to have a declaration for something in order to compile a file into an object file, expecting that the linker can find the definition from another file. If no source file ever defines a symbol, but it is declared, you will get errors at link time complaining about undefined symbols.
- ⌚ **There can be only one definition of the same variable, but many declarations of it (anywhere it is used)**

DIFFERENCE

- ⌚ A variable *declaration* says, "there is a variable with the following name and type in the program".
- ⌚ A variable *definition* says, "Dear Mr. Compiler, please allocate memory for a variable with the following name and type now."

EXAMPLES

```
int func();
```

```
int main()
{
    int x = func();
}
```

func is declared
x is defined

x is defined

```
int x;
int main()
{
    x = 3;
}
```

```
int func()
{
    int x= 3;
}
```

x is defined

Of course, since a definition is also a declaration, they are also declared there

STORAGE DURATION

STORAGE CLASS SPECIFIERS

- ⌚ A ***storage class specifier*** in a declaration modifies the **storage duration** of the corresponding objects
- ⌚ The storage duration of an object can be **automatic**, **static**, or **dynamic**
- ⌚ **auto** is the storage specifier for automatic, and **static** is the storage specifier for static duration; **dynamic** is then using dynamic memory allocation/deallocation

AUTO

- ⌚ **auto**: Objects declared with the auto specifier have automatic storage duration.
- ⌚ This specifier is permissible only in object declarations within a function.
- ⌚ In ANSI C, objects declared within a function have automatic storage duration by default, and **the auto specifier is archaic**.

auto is not used in programs...

```
int main(void) {  
    int a;  
    {  
        int count;  
        auto int month;  
    }  
    a++;  
}
```

REGISTER

- ⌚ **register:** You can use the specifier register when declaring objects with automatic storage duration.
- ⌚ The register keyword is a hint to the compiler that the object should be made as quickly accessible as possible—ideally, by storing it in a CPU register.
- ⌚ However, the compiler may treat some or all objects declared with register the same as ordinary objects with automatic storage duration.
- ⌚ In any case, programs must not use the address operator on objects declared with the register specifier.

EXAMPLE

```
{  
    register int miles;  
}
```

```
{  
    register int miles;  
int* p = &miles;  
}
```

variable miles is not in memory. It is in a register

STATIC

- ⌚ Static: a variable identifier defined with the specifier **static** has static storage duration.
- ⌚ It is created as soon as the program starts, and destroyed (removed from memory) only when the program ends.
- ⌚ Global variables automatically have static storage duration.

EXAMPLE

```
int x= 0;  
int main()  
{  
    x = 3;  
}
```

This is the definition of a global variable (x), which has static storage duration

```
int main()  
{  
    static int x = 3;  
}
```

This is the definition of a local variable (x), which has static storage duration, because I used storage specifier static

SCOPE AND STORAGE DURATION

- ⌚ There are two separate concepts here:
 - ✓ *scope*, which determines where a name can be accessed, and
 - ✓ *storage duration*, which determines when a variable is created and destroyed.
- ⌚ *Automatic* variables (pedantically, variables with *automatic storage duration*) are local variables whose lifetime ends when execution leaves their scope, and are recreated when the scope is reentered.
- ⌚ *Static* variables (pedantically, variables with *static storage duration*) have a lifetime that lasts until the end of the program. If they are local variables, then their value persists when execution leaves their scope.

EXAMPLE 1

```
void f() {  
    int i;  
    i = 1; // OK: in scope  
}  
void g() {  
    i = 2; // Error: not in scope  
}
```

Local variables (pedantically, variables with *block scope*) are only accessible within the block of code in which they are declared: automatic storage duration

```
int i;  
void f() {  
    i = 1; // OK: in scope  
}  
void g() {  
    i = 2; // OK: still in scope  
}
```

Global variables (pedantically, variables with *file scope*) are accessible at any point after their declaration: static storage duration

EXAMPLE 2

```
for (int i = 0; i < 5; ++i) {  
    int n = 0;  
    printf("%d ", ++n);  
}
```

// prints 1 1 1 1 1 - the previous value is lost
// n has automatic storage duration
// it is created anew for every loop iteration
// and destroyed at the end

```
for (int i = 0; i < 5; ++i) {  
    static int n = 0;  
    printf("%d ", ++n);  
}
```

// prints 1 2 3 4 5 - the value persists
// n has static storage duration

EXAMPLE 3

```
for (int i = 0; i < 5; ++i) {  
    static int n = 0;  
    printf("%d ", ++n);  
}  
printf("%d ", n);
```

// error: out of scope

Being forever in memory does not mean that a variable is always accessible: it is accessible in its scope

EXAMPLE 4

```
#include<stdio.h>
void func(void);
```

```
int count=10;
```

```
int main() {
    while (count-->0)
        func();
}
```

```
void func( void )
{
    static int i = 5;
    i++;
    printf("i is %d and count is %d\n", i, count);
}
```

Static storage is the default for
variables outside functions
(global variables)

i is 6 and count is 9
i is 7 and count is 8
i is 8 and count is 7
i is 9 and count is 6
i is 10 and count is 5
i is 11 and count is 4
i is 12 and count is 3
i is 13 and count is 2
i is 14 and count is 1
i is 15 and count is 0

EXAMPLE 5

```
#include<stdio.h>
void func(void);
```

```
int count=10;
```

```
int main() {
    while (count-->0)
        func();
    i++;
}
```

i static but not visible in main

```
void func( void )
{
    static int i = 5;
    i++;
    printf("i is %d and count is %d\n", i, count);
}
```

MacBook-Francesco:Programmi francescosantini\$ gcc -std=c99 -o test test.c
test.c:11:4: error: use of undeclared identifier 'i'

```
    i++;
    ^
```

1 error generated.

EXAMPLE 6

```
#include<stdio.h>
void func(void);
```

```
int main() {
    while (count--)
        func();
}
```

count visible from here



```
int count=10;

void func( void )
{
    static int i = 5;
    i++;
    printf("i is %d and count is %d\n", i, count);
}
```

```
MacBook-Francesco:Programmi francescosantini$ gcc -std=c99 -o test test.c
test.c:8:11: error: use of undeclared identifier 'count'
```

```
    while (count--)
           ^
```

1 error generated.

STATIC STORAGE

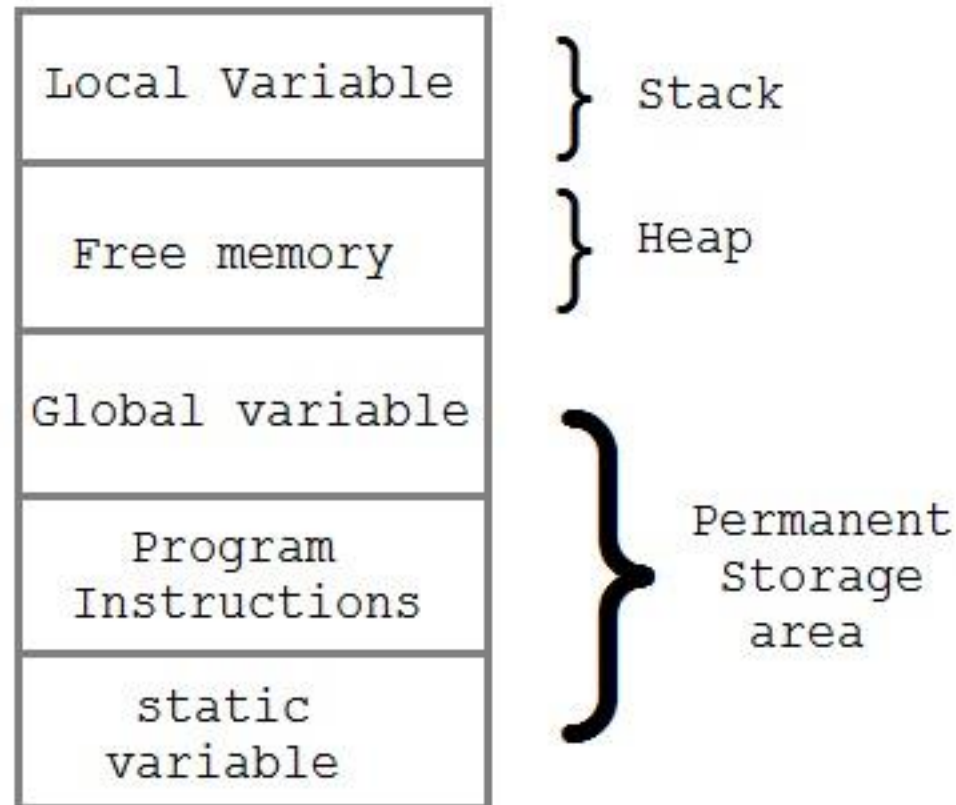
- ⌚ Global variables, or variables within a function and with the storage class specifier **static**, have static storage duration.
- ⌚ All objects with static storage duration are initialized to 0 automatically.
- ⌚ Objects with automatic storage duration are not initialized to 0 by default, their value is undefined
- ✓ It is better to always initialize variables to 0 (especially variables with automatic storage duration)

```
int main() {  
    static int a;    // initialized to 0  
}
```

```
int main() {  
    int a;    // not initialized  
}
```

DIFFERENT KINDS OF MEMORIES

DIFFERENT ZONES IN MEMORY



All your variables (and instructions) are in memory

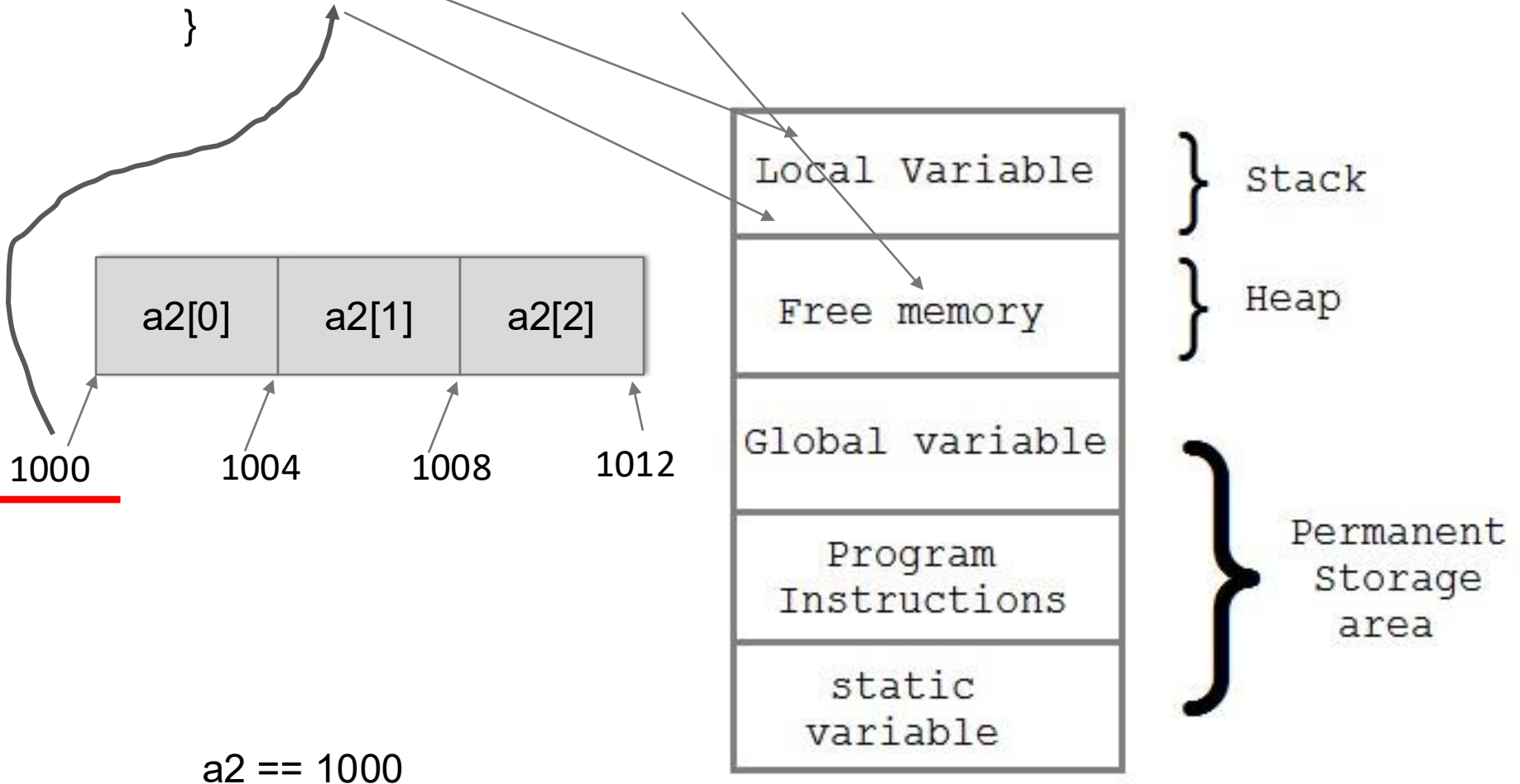
CHARACTERISTICS

- ⌚ Permanent area contains both global and static variables (and instructions): variables with static storage duration.
 - ✓ **Such variables are created when the program starts and destroyed when the program ends**
- ⌚ Local variables are memorized in the stack: it contains variables with automatic storage duration.
 - ✓ **Their duration in memory begins when their definition is encountered and ends at the end of their scope**
- ⌚ The memorization of objects in the heap area is fully under the control of the programmer
 - ✓ **Their duration in memory starts when a programmer allocates it (e.g. `malloc()`), and ends when a programmer deallocates it (`free()`)**

EXAMPLE

```
malloc(3 * sizeof(int));
```

```
int main () {  
    int a1[3];  
    int * a2 = (int*) malloc(3 * sizeof(int));  
}
```



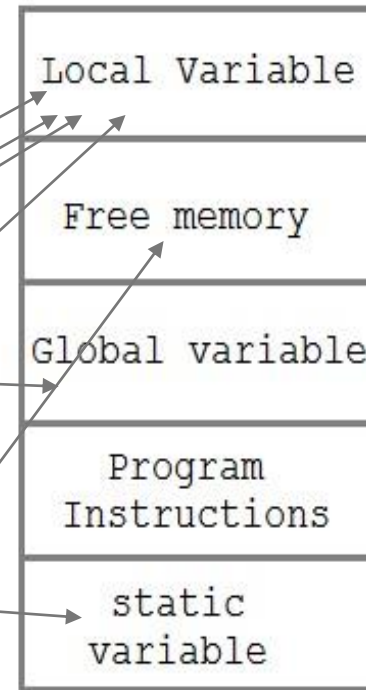
EXAMPLE

```
#include <stdio.h>
```

```
int size= 5;
```

```
void fun(int* b, int n) {  
    for (int i = 0; i < n; ++i ) {  
        static int c= 0;  
        b[i] = ++c;  
    }  
}
```

```
int main(){  
    int* a = (int *) malloc(sizeof(int) * size);  
    fun(a, size);  
    fun(a, size);  
}
```



size
c
...
...
a
...
a[0]
a[1]
a[2]
a[3]
a[4]

EXAMPLE (CONT.)

```
#include <stdio.h>
```

```
int size= 5;
```

```
void fun(int* b, int n) {  
    for (int i = 0; i < n; ++i ) {  
        static int c= 0;  
        b[i] = ++c;  
    }  
}
```

a[0] 6	a[1] 7	a[2] 8	a[3] 9	a[4] 10
-----------	-----------	-----------	-----------	------------

```
int main(){  
    int* a = (int *) malloc(sizeof(int) * size);  
    fun(a, size);  
    fun(a, size);  
    free(a);  // REMEMBER TO FREE ALLOCATED MEMORY  
}
```

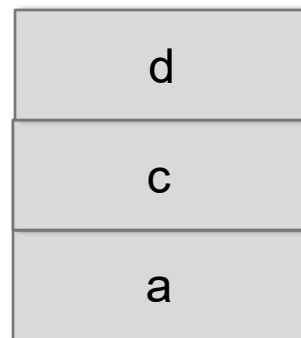
STACK AND FUNCTIONS

CALL STACK

- ④ In computer science, a **call stack** is a stack data structure that stores information about the active subroutines of a computer program.
- ④ This kind of stack is also known as an execution stack, program stack, control stack, run-time stack, or machine stack, and is often shortened to just "the stack".
- ④ Since the call stack is organized as a stack.

STACK

- ④ In computer science, a stack is an abstract data type that serves as a collection of elements, with two principal operations: **push**, which adds an element to the collection, and **pop**, which removes the most recently added element that was not yet removed.
- ④ The order in which elements come off a stack gives rise to its alternative name, **LIFO** (for last in, first out).



push

CALL TO FUNCTIONS AND STACK

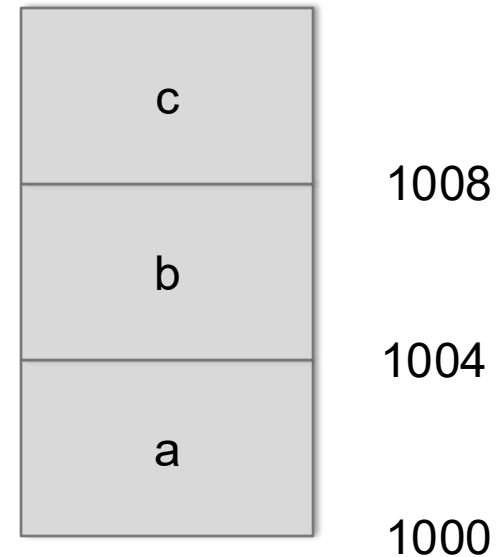
```
include<stdio.h>
```

```
bye f2  
bye f1  
bye main
```

```
void f2() {  
    int c;  
    puts("bye f2");  
}
```

```
void f1() {  
    int b= 0;  
    f2();  
    puts("bye f1");  
}
```

```
int main() {  
    int a= 0;  
    f1();  
    puts("bye main");  
}
```



EXAMPLE 2

```
include<stdio.h>
```

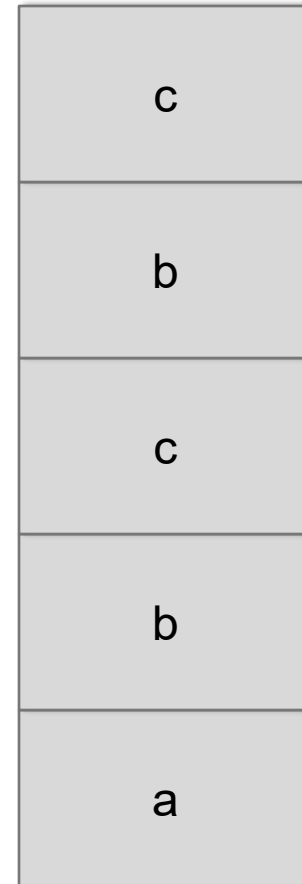
```
void f1();
```

```
void f2() {  
    int c;  
    f1();  
    puts("bye f2");  
}
```

```
void f1() {  
    int b= 0;  
    f2();  
    puts("bye f1");  
}
```

```
int main() {  
    int a= 0;  
    f1();  
    puts("bye main");  
}
```

.
.
.



MacBook-Francesco:Programmi francescosantini\$./test
Segmentation fault: 11

STACK OVERFLOW

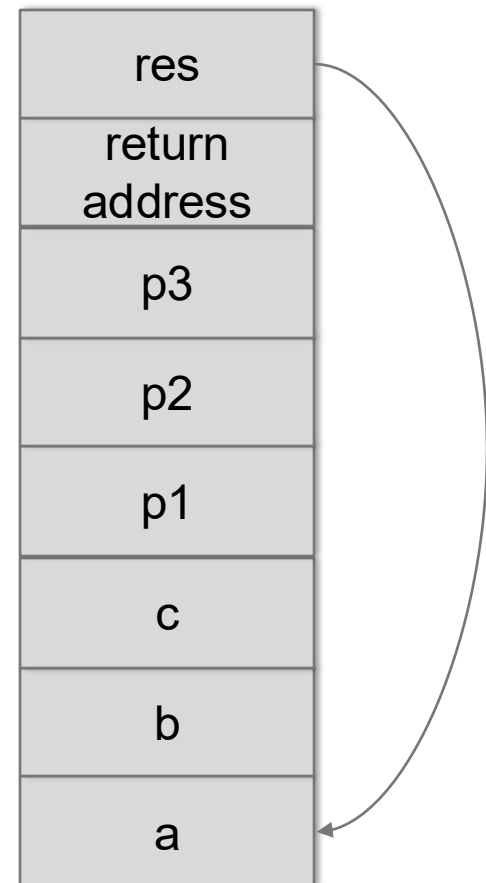
- @ Also the size of the stack is limited (as the heap)
- @ It cannot grow infinite
- @ In the example before, the stack grows up until it reaches a zone of memory that it cannot access anymore
 - ✓ Segmentation fault

ALSO MEMORY ADDRESS

The stack grows from bottom to top, heap is different

```
int fun(int p1, int p2, int p3) {  
    int res= 0;  
    res= p1 + p2 + p3;  
    return res;  
}
```

```
int main() {  
    int a= 4, b= 5, c= 7;  
    a= fun(a,b,c);  
}
```



SU LIBRO E RIFERIMENTI

@ Sezione 5.12

@ Sezione 5.7

@ Altri riferimenti

✓ <https://shorturl.at/fknPY>

✓ <https://profrizzo.altervista.org/memoria-ram-stack-vs-heap-memoria-statica-vs-memoria-dinamica/>

✓ https://www.riochierego.it/docs/10-Allocazione_dinamica_della_memoria.pdf